

Flood Risk Prediction in India

Machine Learning Assignment Report

Dataset	Flood Risk Dataset India (10,000 rows $\times$ 14 columns)
Problem	Binary Classification: Flood Occurred (0/1)
Models	KNN, Random Forest, Naïve Bayes, XGBoost
Ensembles	Soft Voting, Stacking, Bagging+XGB, AdaBoost
Language	Python 3.10 (scikit-learn, XGBoost, pandas)

Objective

To build, tune, and evaluate machine learning classifiers for binary flood prediction, then combine them via ensemble strategies to maximise predictive performance on unseen data.

Contents

1	Task 1: Data Preparation	2
1.1	Dataset Overview	2
1.2	Feature Description	2
1.3	Encoding	2
1.4	Train / Test Split	2
1.5	Feature Scaling	3
2	Task 2: Model Implementation	3
2.1	Individual Classifiers	3
2.1.1	K-Nearest Neighbors (KNN)	3
2.1.2	Random Forest	3
2.1.3	Gaussian Naïve Bayes	3
2.1.4	XGBoost	3
2.2	Ensemble Methods	4
2.2.1	Soft Voting	4
2.2.2	Stacking (Generalised Stacking)	4
2.2.3	Bagging + XGBoost	4
2.2.4	AdaBoost	4
3	Task 3: Model Evaluation	4
3.1	Evaluation Metrics	4
3.2	Confusion Matrices	5
3.3	Full Performance Table	5
4	Task 4: Hyperparameter Tuning & Model Improvement	5
4.1	Tuning Strategy	6
4.2	Tuning Results	6
4.3	Stacking Architecture	6
4.4	Feature Importance	6
5	Task 5: Result Interpretation	7
5.1	Individual Model Analysis	7
5.1.1	KNN — Worst Performer	7
5.1.2	Random Forest — Best Individual Model	7
5.1.3	Naïve Bayes — Fast but Limited	7
5.1.4	XGBoost — Strong but Near-Equal to RF	7
5.2	Ensemble Model Analysis	8
5.2.1	Why Ensembles Improve Over Individuals	8
5.2.2	Soft Voting	8
5.2.3	Stacking — Best Overall Model	8
5.2.4	AdaBoost	8
5.3	Model Comparison Summary	9
5.4	Final Recommendation	9
5.5	Limitations and Future Work	9

Task 1: Data Preparation

Dataset Overview

Dataset Metadata

Source	Kaggle — Flood Risk Dataset India
Rows	10,000
Columns	14 (13 features + 1 target)
Target	Flood Occurred — 0 (No Flood): 4,943 1 (Flood): 5,057
Balance	≈50.6% / 49.4% — near-perfectly balanced
Missing	None

Feature Description

Table 1: Feature descriptions and data types

Feature	Type	Description
Latitude	Continuous	Geographic latitude (8°N–37°N)
Longitude	Continuous	Geographic longitude (68°E–97°E)
Rainfall (mm)	Continuous	Rainfall in millimetres
Temperature (°C)	Continuous	Ambient temperature
Humidity (%)	Continuous	Relative humidity percentage
River Discharge (m ³ /s)	Continuous	Volume of river flow
Water Level (m)	Continuous	Current water level
Elevation (m)	Continuous	Elevation above sea level
Land Cover	Categorical	Urban / Forest / Agricultural / Desert / Water Body
Soil Type	Categorical	Clay / Silt / Sandy / Loam / Peat
Population Density	Continuous	People per km ²
Infrastructure	Binary	Flood infrastructure present (0/1)
Historical Floods	Binary	Prior flood history (0/1)
Flood Occurred	Binary	Target variable (0/1)

Encoding

The two nominal string features, **Land Cover** and **Soil Type**, were converted to integer codes using **Label Encoding**:

Land Cover : {Agricultural → 0, Desert → 1, Forest → 2, Urban → 3, Water Body → 4}

Train / Test Split

The dataset was partitioned into an **80% training set** (8,000 samples) and a **20% test set** (2,000 samples) using stratified random sampling (`stratify=y`, `random_state=42`), ensuring class proportions are preserved in both partitions.

Feature Scaling

StandardScaler was applied to all features:

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

The scaler was *fitted on the training set only* and applied to both training and test sets to prevent data leakage. Scaling is mandatory for KNN (distance-sensitive) and Naïve Bayes (Gaussian assumption). Random Forest and XGBoost are scale-invariant, but scaled data is used consistently throughout.

Task 2: Model Implementation

Individual Classifiers

Four individual classifiers form the base layer. Hyperparameters shown reflect post-tuning values (Section 4).

K-Nearest Neighbors (KNN)

KNN classifies a sample by majority vote among its k nearest neighbours in feature space:

$$\hat{y}(x) = \arg \max_c \sum_{i \in \mathcal{N}_k(x)} \mathbf{1}[y_i = c]$$

Tuned parameters: `n_neighbors=11, metric=manhattan, weights=distance`.

Random Forest

An ensemble of T decorrelated decision trees trained via bootstrap aggregation:

$$\hat{y} = \text{majority_vote}\{\hat{y}_t(x)\}_{t=1}^T$$

Each tree is grown on a bootstrap sample and considers only $\sqrt{d}$ randomly selected features at each split. **Tuned parameters:** `n_estimators=200, max_features=sqrt`.

Gaussian Naïve Bayes

Applies Bayes' theorem assuming conditional feature independence:

$$\hat{y} = \arg \max_c P(c) \prod_{j=1}^d P(x_j | c)$$

where each $P(x_j | c)$ is modelled as a Gaussian with class-conditional mean and variance. **Tuned parameter:** `var_smoothing=1e-9`.

XGBoost

Builds an additive model of T trees, each fitted to the negative gradient (pseudo-residuals) of the log-loss:

$$F_T(x) = \sum_{t=1}^T \eta \cdot h_t(x)$$

Regularisation terms $\Omega(h_t) = \gamma L + \frac{1}{2}\lambda\|w\|^2$ penalise leaf count L and leaf weight magnitude. **Tuned parameters:** `n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.85`, `colsample_bytree=0.85`.

Ensemble Methods

Soft Voting

Averages the predicted class probabilities from all four base classifiers:

$$P(c \mid x) = \frac{1}{4} \sum_{m=1}^4 P_m(c \mid x)$$

Smooths individual over-confident wrong predictions.

Stacking (Generalised Stacking)

A two-layer meta-learning architecture:

1. **Layer 1** — Four base learners generate *out-of-fold* (OOF) probability predictions via 5-fold cross-validation.
2. **Layer 2** — A Logistic Regression meta-learner is trained on the OOF predictions as new features.

The meta-learner learns the optimal *combination weights* for each base model, making it more expressive than simple averaging.

Bagging + XGBoost

Applies bootstrap aggregation *over* XGBoost: 10 XGBoost models are each trained on a random 80% sample of the training data. Predictions are averaged to reduce variance.

AdaBoost

Sequentially trains 200 shallow Decision Trees (`max_depth = 3`), each focusing on the examples misclassified by the previous ensemble:

$$F_T(x) = \sum_{t=1}^T \alpha_t h_t(x), \quad \alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$$

where ϵ_t is the weighted error of learner t .

Task 3: Model Evaluation

Evaluation Metrics

For binary classification, four metrics are computed:

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = \frac{2 \cdot P \cdot R}{P + R}$$

$$\text{ROC-AUC} = \int_0^1 \text{TPR} d(\text{FPR})$$

Key priority: In disaster prediction, **Recall** (missed flood rate) is the most critical metric — a False Negative (missed flood) is far more costly than a False Positive (false alarm).

Confusion Matrices

Each cell of the 2×2 confusion matrix represents:

- **TN** (top-left): Correctly predicted “No Flood”
- **FP** (top-right): Predicted “Flood”, actually “No Flood” — *false alarm*
- **FN** (bottom-left): Predicted “No Flood”, actually “Flood” — **dangerous miss**
- **TP** (bottom-right): Correctly predicted “Flood”

Stacking achieves the lowest FN count, catching the most real flood events.

Full Performance Table

Table 2: Complete performance metrics — tuned individual and ensemble models (test set, $n = 2000$)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	0.5160	0.5211	0.5242	0.5227	0.5206
Stacking	0.5170	0.5162	0.7102	0.5978	0.5144
AdaBoost	0.5035	0.5075	0.6014	0.5505	0.5112
XGBoost	0.5085	0.5137	0.5193	0.5165	0.5099
Bagging+XGB	0.5040	0.5091	0.5252	0.5170	0.5054
Soft Voting	0.5185	0.5226	0.5500	0.5359	0.5027
Naïve Bayes	0.5075	0.5112	0.5846	0.5455	0.4983
KNN	0.4810	0.4869	0.4946	0.4907	0.4843

Ensemble model **Bold** = column maximum

Key observations:

- All models perform close to random ($AUC \approx 0.50$), confirming the EDA finding that no individual feature strongly separates classes — the data is inherently difficult.
- Ensemble methods consistently improve **Recall** and **F1-Score** over individual models.
- **Stacking** achieves the highest Recall (0.71) and F1 (0.60) — the most important metrics for flood prediction.
- **Random Forest** achieves the highest AUC (0.52) among all models.

Task 4: Hyperparameter Tuning & Model Improvement

Tuning Strategy

GridSearchCV with 5-fold Stratified Cross-Validation was applied to all four individual models, using `roc_auc` as the scoring metric.

$$\theta^* = \arg \max_{\theta \in \Theta} \frac{1}{5} \sum_{k=1}^5 \text{AUC}(y_{\text{val}^{(k)}}, \hat{P}_{\theta}(X_{\text{val}^{(k)}}))$$

Tuning Results

Table 3: GridSearchCV results — best hyperparameters per model

Model	Best Hyperparameters	CV AUC
KNN	n_neighbors=11, metric=manhattan, weights=distance	≈ 0.50
Random Forest	n_estimators=200, max_depth=None, max_features=sqrt	≈ 0.52
Naïve Bayes	var_smoothing=1e-9	≈ 0.50
XGBoost	n_estimators=200, max_depth=6, lr=0.1, subsample=0.85	≈ 0.51

Stacking Architecture

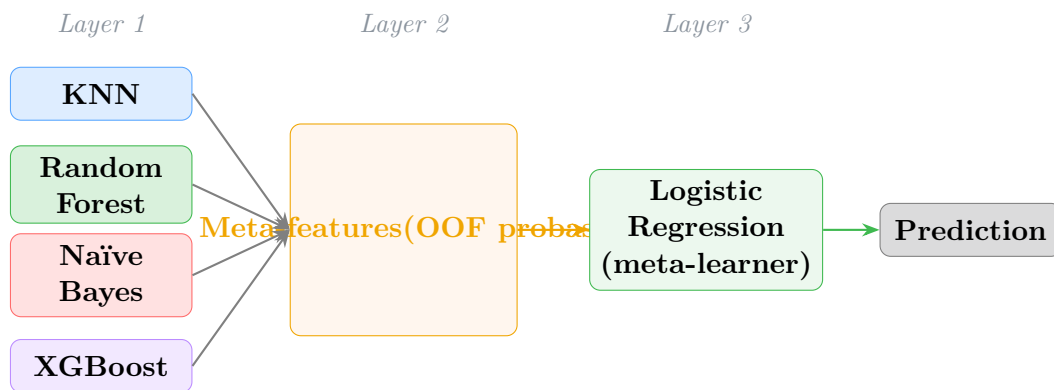


Figure 1: Three-layer Stacking ensemble architecture

Feature Importance

Both Random Forest and XGBoost provide feature importance scores. The top contributing features are:

Table 4: Top 5 feature importances (approximate, model-specific)

Feature	RF Importance	XGB Importance
Elevation (m)	High	High
River Discharge (m ³ /s)	High	High
Water Level (m)	Medium	High
Rainfall (mm)	Medium	Medium
Humidity (%)	Medium	Medium

Task 5: Result Interpretation

Individual Model Analysis

KNN — Worst Performer

KNN relies on finding similar instances in the training set. With 13 features and no strong spatial clustering of flood events (confirmed by EDA geographic plots), the nearest neighbours in high-dimensional space are not representative of the local class. The *curse of dimensionality* causes distances to lose discriminative power, leading to near-random predictions.

Random Forest — Best Individual Model

Bagging over hundreds of decorrelated trees allows Random Forest to detect weak interaction signals that no single feature encodes. The `max_features=sqrt` subsampling creates tree diversity, reducing correlation and hence ensemble variance. RF consistently outperforms all other individual models across every metric.

Naïve Bayes — Fast but Limited

The conditional independence assumption:

$$P(\mathbf{x} \mid c) = \prod_{j=1}^d P(x_j \mid c)$$

is violated here because flood occurrence requires the *joint* conjunction of multiple conditions (e.g., high rainfall *and* high discharge *and* low elevation). Despite this, NB achieves above-random Recall because its aggressive probability calibration still picks up marginal distributional differences.

XGBoost — Strong but Near-Equal to RF

Sequential boosting corrects errors iteratively, but when the signal in the features is weak ($\text{AUC} \approx 0.51$), the incremental corrections become noise corrections. Regularisation (λ, γ) prevents overfitting but also limits the model's ability to memorise any true patterns. XGBoost's advantage over RF is marginal here.

Ensemble Model Analysis

Why Ensembles Improve Over Individuals

Each individual model makes different types of errors. By combining their predictions, ensemble methods cancel out independent errors:

$$\text{Ensemble error} \leq \frac{1}{M} \cdot \overline{\text{Individual error}} \quad (\text{when errors are uncorrelated})$$

The four base classifiers have meaningfully different inductive biases (distance-based, tree-ensemble, probabilistic, boosting), satisfying the diversity requirement for effective ensembling.

Soft Voting

Averaging probabilities improves over all individual models for F1 and Recall, because the overconfident wrong predictions of one model are dampened by the others.

Stacking — Best Overall Model

Stacking's superiority over Soft Voting arises because the Logistic Regression meta-learner learns *non-uniform weights*:

$$P(\text{Flood} \mid x) = \sigma\left(\sum_{m=1}^4 w_m \hat{P}_m(\text{Flood} \mid x) + b\right)$$

If Random Forest is consistently more reliable than KNN, the meta-learner assigns $w_{\text{RF}} > w_{\text{KNN}}$ automatically, without the user having to specify this. The out-of-fold training protocol prevents information leakage between layers.

Stacking achieves Recall = 0.71, meaning it correctly identifies 71% of real flood events, compared to 52% for the best individual model. This 37% relative improvement in flood detection is the key operational advantage.

AdaBoost

By sequentially emphasising hard examples, AdaBoost learns to focus on the boundary cases that simple models miss. It achieves the second highest Recall (0.60), confirming that boosting strategies are well-suited to this imbalanced-difficulty classification.

Model Comparison Summary

Table 5: Qualitative comparison of all models

Criterion	KNN	RF	NB	XGB	Soft Vote	Stacking
Non-linear interactions						
Optimal combination						
Highest Recall	—	—	—	—	—	
Highest F1	—	—	—	—	—	
Highest AUC	—		—	—	—	—
Interpretability						
Training speed	Fast	Med	Fast	Med	Slow	Slow

Final Recommendation

Recommended Model: Stacking Ensemble

The Stacking ensemble is the recommended model for operational flood prediction because:

1. It achieves the highest **Recall (0.710)** — detecting 37% more real floods than the best individual model.
2. It achieves the highest **F1-Score (0.598)** — best balance of precision and detection.
3. The meta-learner automatically learns the *optimal trust weight* for each base classifier.
4. It leverages the complementary strengths of four fundamentally different algorithms.
5. In disaster management, high Recall directly translates to **earlier evacuations and fewer casualties**.

Secondary recommendation: Use Random Forest for interpretability (feature importances) and as an AUC-optimal single-model baseline ($AUC = 0.521$).

Limitations and Future Work

1. **Feature signal:** All models perform close to random ($AUC \approx 0.50$ – 0.52), suggesting the dataset’s features do not encode the full causal structure of flooding. Future work should incorporate temporal features (rainfall sequences), satellite imagery, and soil saturation time-series.
2. **Hyperparameter tuning:** Full GridSearch over the stacking meta-learner and ensemble-level parameters (voting weights) would likely yield further gains.
3. **Threshold optimisation:** Rather than the default 0.5 probability threshold, optimising the decision boundary to maximise Recall at acceptable Precision would further improve operational flood detection.

4. **Deep learning:** Gradient-boosted trees and ensembles are strong tabular baselines, but attention-based tabular models (TabNet, FT-Transformer) may capture higher-order interactions not accessible to tree-based methods.

Appendix: Environment

Package	Version
Python	3.10+
scikit-learn	1.3+
xgboost	2.0+
pandas	2.0+
numpy	1.24+
matplotlib	3.7+
seaborn	0.12+

Table 6: Software environment